

Can a PDDL 3.1 Model Be Completely Converted to POWL v2 with Concurrency Maximized?

A guided walk from college statistics to the answer

Sean Chatman

July 14, 2026

Abstract

This tutorial assumes one course in college statistics — you know what a sample space is, what it means to partition data into groups, how to count permutations, and what log does — and builds, step by step, to a precise answer to the question in the title. The short version: the question is really *three* questions, and they have three different answers. *Can it be converted?* Yes, for a precisely characterized class (“separable” structures), proven by Kourani, Park & van der Aalst (2026), and empirically that class covered 100% of a benchmark of 1,493 industrial process models — but explicit counterexamples exist outside the class. *Completely?* Yes on that class: the conversion provably preserves the exact set of possible execution sequences. *With concurrency maximized?* This is the open frontier: “maximized” has an exact mathematical meaning (the transformation must erase *all* accidental ordering and *only* accidental ordering), it is formalized in Lean 4 as the MFW *crown theorem*, and it is currently a conjecture with substantial proven scaffolding — plus one honest complication: PDDL 3.1’s clocks and resource constraints mean that maximal *causal* concurrency is not the same as maximal *executable* concurrency.

Contents

1	The question, taken apart	1
2	Stage 1: A planning model is a sample space of stories	2
2.1	From statistics to plans	2
3	Stage 2: POWL v2 is a grouped, hierarchical description	3
3.1	Why we want a second representation	3
3.2	The statistical reading	3
4	Stage 3: Accidental order, and how to measure it	3
4.1	Independence of actions	3
4.2	Trace equivalence: same story, different typing order	4
4.3	Counting the redundancy: serialization entropy	4
4.4	The honest complication: clocks	4

5	Stage 4: The conversion theorem — what is actually proven	5
5.1	Workflow nets: the neutral middle ground	5
5.2	The structural duality	5
5.3	The algorithm and the two theorems	5
5.4	The boundary is real: counterexamples	6
5.5	How much of reality is separable?	6
6	Stage 5: Maximality — the kernel and the crown theorem	6
6.1	The kernel: what the conversion merges	6
6.2	What is already proven around the conjecture	7
6.3	Diagnostics: how you would check maximality on real output	7
7	The verdict	8

1 The question, taken apart

“Can a PDDL 3.1 be completely converted to POWL v2 so that concurrency is maximized?” hides three separate claims. It helps to name them now, because they have different mathematical statuses:

- Q1. Convertibility.** Does every PDDL 3.1 planning model have *some* POWL v2 representation at all?
- Q2. Completeness.** When a representation exists, does it capture *exactly* the same behavior — nothing added, nothing lost?
- Q3. Maximality.** Among the correct representations, does the conversion find the one with the *most* concurrency — every ordering constraint that can be dropped, is dropped?

By the end you will be able to state the answers precisely. The road has five stages: (§2) what a PDDL 3.1 model is, statistically; (§3) what POWL v2 is; (§4) the key idea — *accidental order* — and how to count it; (§5) the proven conversion result and its counterexamples; (§6) the maximality question and the crown theorem. §7 assembles the verdict.

2 Stage 1: A planning model is a sample space of stories

2.1 From statistics to plans

In statistics you start with a sample space Ω : the set of all outcomes an experiment could produce. A *planning model* is the same idea where an “outcome” is an entire *story*: a sequence of actions, each with a timestamp, that starts in a known situation and ends with a goal achieved.

PDDL 3.1 (Planning Domain Definition Language, version 3.1) is a standard language for writing down such models. A PDDL 3.1 model Π specifies:

- a set of **states** S (situations the world can be in) and a designated initial state s_0 ;
- a set of **actions** A , and a *transition function* $\delta(s, a)$ saying what state results from doing action a in state s — or that a is impossible there (like a conditional probability table, except deterministic: each defined entry has exactly one result);

- a **goal**: which final states count as success;
- **temporal constraints**: actions take time, and there are deadlines and duration windows;
- **numeric constraints**: quantities like fuel that actions consume and preconditions test;
- **trajectory constraints**: rules about the whole story, e.g. “the door must *always* be closed” or “the alarm must fire *sometime*.”

Definition 2.1 (Behavior, informally). A *behavior* is a pair $b = (\mathbf{e}, \mathbf{t})$: a list of actions $\mathbf{e}$ and an equal-length list of timestamps $\mathbf{t}$. A behavior is *lawful* if (i) each action is possible when its turn comes, (ii) the story ends in a goal state, and (iii) all temporal, numeric, and trajectory constraints hold.

Definition 2.2 (The behavioral phase space). $\mathcal{P}(\Pi)$ = the set of all lawful behaviors of Π . This is our sample space. Every question in this tutorial is a question about $\mathcal{P}(\Pi)$.

Example 2.3. Two trucks, two packages, locations A and C . Goal: both packages at C . One lawful behavior: load p_1 on truck 1, load p_2 on truck 2, drive truck 1, drive truck 2, unload both. Another lawful behavior: the same actions with the two loads in the opposite order. Both stories succeed. Keep this example in mind — the whole subject turns on the relationship between those two stories.

Checkpoint 2.4. $\mathcal{P}(\Pi)$ is typically enormous: if k actions could go in any order, that alone contributes up to $k!$ behaviors (permutations — the counting you know from statistics). Most of that size is bookkeeping, not substance. Quantifying “bookkeeping” is Stage 3.

3 Stage 2: POWL v2 is a grouped, hierarchical description

3.1 Why we want a second representation

A list of a million stories is not understanding. What we want is a *model of the process*: a compact diagram saying “these steps happen in some order, these are alternatives, this part repeats, these two threads run side by side.” This is exactly the move statistics makes when it replaces raw data with a fitted model.

POWL v2 (Partially Ordered Workflow Language, version 2.0; Kourani & van Zelst) is such a language. A POWL model is built recursively — small models nested inside bigger ones — from two constructs:

- Definition 3.1** (POWL v2, informally).
- A single activity is a POWL model (the base case).
 - A **partial order** over n submodels is a POWL model: all n submodels execute, subject to stated precedence constraints ($i \prec j$ means “ i must finish before j starts”). Pairs with no constraint may run *concurrently*, in either order or overlapping.
 - A **choice graph** over n submodels is a POWL model: a directed graph with a start and an end marker; each path from start to end is one allowed way through, executing the submodels it visits in sequence. Choice graphs express decisions *and* cycles (a loop is a path that revisits a node).

The *meaning* (formally, the *language* $\mathcal{L}(\psi)$) of a POWL model ψ is the set of action sequences it permits: for a partial order, all interleavings of the children’s sequences consistent

with $\prec$ (the *order-preserving shuffle*); for a choice graph, the concatenations along each start-to-end path.

Checkpoint 3.2. Note the division of labor, because the conversion algorithm in Stage 4 exploits it exactly: *partial orders carry all concurrency; choice graphs carry all decisions and loops.* Each construct is forbidden from doing the other’s job.

3.2 The statistical reading

A partial order with few constraints is like reporting “these variables are independent”: it licenses many joint outcomes from a short description. A total order (every pair constrained) licenses exactly one sequence. So **concurrency in the model = permissiveness = how many lawful interleavings one diagram covers**. “Maximize concurrency” will mean: *use the fewest precedence constraints that still tell the truth.*

4 Stage 3: Accidental order, and how to measure it

This is the conceptual heart of the whole subject.

4.1 Independence of actions

In Example 2.3, “load p_1 on truck 1” and “load p_2 on truck 2” touch different trucks and different packages. Doing them in either order leads to the same state, neither disables the other, no shared fuel is overdrawn, no rule about the whole story is affected. Call such a pair of actions **independent**, written $(a, b) \in I$. Formally (this is the checkable definition used in the Lean formalization), independence requires five things: effects commute, preconditions stay stable, invariants stay stable, numeric flows are compatible, trajectory constraints are preserved.

This is a direct cousin of statistical independence: two factors are independent when the outcome doesn’t depend on the design order.

4.2 Trace equivalence: same story, different typing order

Definition 4.1 (Mazurkiewicz trace equivalence). Two action sequences are *trace equivalent*, written $u \sim_I v$, if you can turn one into the other by repeatedly swapping *adjacent independent* pairs.

In Example 2.3, the two stories are trace equivalent: swap the two adjacent loads. They are not two behaviors of the process; they are one behavior of the process written down twice. The difference between them is **accidental order** — order imposed by the act of writing a sequence, not by causality. (This is a classical construction: Mazurkiewicz 1977; Diekert & Rozenberg 1995. That it is a genuine equivalence relation — reflexive, symmetric, transitive — is **PROVEN** in the Lean formalization.)

4.3 Counting the redundancy: serialization entropy

Group the sequences into equivalence classes (exactly like grouping raw observations by treatment in ANOVA). Each class corresponds to one *causal partial order* P : keep $a \prec b$ only when

a and b are dependent and a came first. The class then contains every *linear extension* of P — every total ordering consistent with $\prec$. Write $e(P)$ for the number of linear extensions.

Definition 4.2 (Serialization entropy).

$$H_{\text{ser}}(P) = \log e(P).$$

You have seen this shape before: $\log(\text{number of equally-good possibilities})$ is exactly Shannon’s entropy for a uniform distribution. $H_{\text{ser}} = 0$ means the order is fully forced ($e(P) = 1$). Large H_{ser} means the sequence representation is padding one causal object with many redundant serializations. With k mutually independent actions, $e(P) = k!$ and $H_{\text{ser}} = \log k!$ — the redundancy grows *factorially*. This is the precise sense in which a good conversion can achieve enormous compression: PDDL-to-POWL is, potentially, factorial state-space compression by quotienting interleavings.

4.4 The honest complication: clocks

Not every linear extension is temporally feasible. If action a must finish by noon and b can’t start before 3pm, the causal order may permit “ b then a ” while the clock forbids it. Write $T(P, N)$ for the extensions that admit a schedule satisfying the temporal network N , and $H_T(P, N) = \log |T(P, N)|$. Then

$$\underbrace{H_{\text{ser}}(P) - H_T(P, N)}_{\text{temporal restriction gap}} \geq 0$$

measures concurrency that *looks* available causally but is *not* executable. More generally the formalization distinguishes four concurrency notions — causal C_C , temporal C_T , resource C_R , and **executable** $C_E = C_C \cap C_T \cap C_R$. Keep this in your pocket for the final verdict: “maximize concurrency” must say *which* concurrency.

Checkpoint 4.3. We now have the target in one sentence. *The ideal conversion maps each PDDL 3.1 behavior to the diagram of its causal partial order — erasing all accidental order (maximal concurrency) and only accidental order (no false merging).*

5 Stage 4: The conversion theorem — what is actually proven

We now examine the machinery for Q1 and Q2, following Kourani, Park & van der Aalst, *Hierarchical Decomposition of Separable Workflow-Nets* (2026).

5.1 Workflow nets: the neutral middle ground

Process behavior is standardly encoded as a *workflow net* (WF-net): a Petri net — boxes (transitions = actions) and circles (places = waiting conditions) with tokens flowing through — having a unique start place, a unique end place, and every node on a path between them. Two hygiene conditions matter: *safe* (no place ever holds two tokens) and *sound* (no dead actions; from any reachable situation, finishing is still possible; the end is reached cleanly). A grounded PDDL model’s reachable behavior can be encoded this way; the WF-net is the neutral, graph-shaped middle ground between PDDL and POWL.

5.2 The structural duality

Inside Petri nets, POWL’s two constructs have exact structural counterparts:

POWL construct	Petri-net pattern	Carries
partial order	<i>marked graph</i> (every place: ≤ 1 in, ≤ 1 out)	concurrency
choice graph	<i>state machine</i> (every transition: ≤ 1 in, ≤ 1 out)	decisions, loops

Definition 5.1 (Separable WF-net). A WF-net is *separable* if it can be built recursively by substituting transitions with subnets that each behave locally as a state machine or a marked graph. Intuition: **decisions and concurrency may nest inside one another to any depth, but may not cross-link** — e.g., no choice whose resolution depends mid-flight on the state of a parallel branch without a synchronization point.

5.3 The algorithm and the two theorems

The conversion algorithm is top-down recursive descent. At each level it tries to split the net either into a partial order over parts (by *hiding conflicts*: pulling every decision entirely inside some part) or into a choice graph over parts (by *hiding concurrency*: pulling every parallel split/join entirely inside some part); it projects the net onto each part, recurses, and reassembles. If neither split makes progress, it stops and reports failure (“fall-through”). An optional preprocessing pass applies language-preserving rewrites (deduplicating places, making XOR splits/joins explicit) that turn some non-separable nets into equivalent separable ones.

Theorem 5.2 (Correctness; Kourani et al. Thm. 5.5). **PROVEN** *If the algorithm converts a safe and sound WF-net N into a POWL model ψ without fall-through, then $\mathcal{L}(N) = \mathcal{L}(\psi)$: the POWL model permits exactly the execution sequences of the original — no behavior lost, none invented.*

Theorem 5.3 (Completeness on the separable class; Thm. 5.6). **PROVEN** *Every safe and sound separable WF-net is successfully converted (the fall-through is never invoked).*

This is the exact sense in which the answer to Q1 and Q2 is *yes with a boundary*: on the separable class, conversion is total and lossless.

5.4 The boundary is real: counterexamples

POWL 2.0 is a strict subclass of sound WF-nets, and the paper exhibits sound — even free-choice — nets that no POWL 2.0 model captures:

- **Long-term dependency**: an early choice between a and b that constrains a much later choice between d and e across intervening structure. The decision logic cannot be encapsulated into any single-entry single-exit block.
- **A loop jumping into a parallel branch**: cyclic behavior that re-enters one thread of a concurrent region, skipping its sibling.
- **Choice and concurrency sharing a synchronization point**: one thread’s decision alters what a parallel thread must synchronize with, mid-flight.

In each case both split attempts provably fail: the partition needed to hide conflicts (or concurrency) violates the single-entry/single-exit requirements. This is the precise content of “decisions and concurrency may nest but not cross-link.”

5.5 How much of reality is separable?

On a benchmark of 1,493 sound workflow nets — 493 industrial models from the SAP R/3 reference set plus 1,000 synthetic models mimicking their complexity — the algorithm (with pre-processing) converted **100%**, and did so faster than prior converters (milliseconds per model). Statistically: the separable class is not everything, but on real business-process logic observed so far, non-separability did not occur. (Usual caveat: a benchmark is a sample, not the population; and planning domains are not business processes — more on this below.)

6 Stage 5: Maximality — the kernel and the crown theorem

Correctness (Q2) says the conversion tells the truth. Maximality (Q3) asks whether it tells the *whole* truth about concurrency. This is the subject of the MFW (Multi Fractal Workflow) formalization in Lean 4 (`ProcInt.MFW`, 16 modules, compiled with zero unproven placeholders in the code — though key statements are stated as conjectures).

6.1 The kernel: what the conversion merges

Let $\tau : \mathcal{P}(\Pi) \rightarrow \mathcal{W}$ be the whole pipeline: lawful behavior in, POWL v2 object out. Two behaviors are merged when $\tau(b_1) = \tau(b_2)$; the set of behaviors mapped to one workflow w is the *fiber* $F_w = \tau^{-1}(w)$. The fibers partition $\mathcal{P}(\Pi)$ — every behavior in exactly one group (**PROVEN**), like a grouping variable partitions a data set. The induced equivalence ($b_1 \equiv_K b_2 \iff \tau(b_1) = \tau(b_2)$) is called the **kernel** of τ .

Now everything from Stage 3 pays off. There are two partitions of the same sample space on the table:

Partition	Comes from
kernel classes (τ -fibers)	the <i>conversion</i> (what got merged)
trace classes ($\sim_I$)	the <i>physics</i> (what is genuinely the same story)

Conjecture 6.1 (The crown theorem). **CONJECTURAL**

$$\tau(b_1) = \tau(b_2) \iff b_1 \sim_I b_2 \quad \text{equivalently} \quad \ker(\tau) = \text{TraceEq}_I.$$

Read the two directions as the two halves of Q3:

- ($\Leftarrow$) *Maximal concurrency*: whenever two behaviors differ only by swapping independent actions, the conversion merges them. No accidental order survives into the diagram.
- ($\Rightarrow$) *No over-merging*: the conversion never merges behaviors that differ causally. Maximality without this direction would be cheating (map everything to one blob — “maximal concurrency,” zero truth).

So “concurrency is maximized” is not a vibe; it is this biconditional. The formalization proves the statement is *non-circular* (**PROVEN**): the left side is defined purely from the converter, the right side purely from PDDL semantics and independence; neither mentions the other. If proved, it is a genuine structural correspondence between an algorithm and the physics of the domain.

6.2 What is already proven around the conjecture

- Both relations are equivalence relations; fibers partition the space and distinct fibers are disjoint (**PROVEN**).
- *Conditional identity*: if the crown holds, then each fiber *is* a trace class, $F_{\tau(b)} = [b]_I$ (**PROVEN** from the hypothesis).
- *Counting*: trace classes correspond to linear extensions of the causal order, $|[b]_I| = e(P_b)$ (**CONJECTURAL** in Lean; classical in concurrency theory under finiteness and distinct-events hypotheses).
- *Entropy collapse*: chaining the above, the information the conversion erases at w , namely $S_\tau(w) = \log |F_w|$ (*fiber entropy* — again log of a count, and provably between 0 and $\log |\mathcal{P}(\Pi)|$), would equal $H_{\text{ser}}(P_b)$ exactly. In words: **if the crown holds, the conversion erases precisely the accidental order — no more, no less — and the amount erased is measurable from the diagram alone** (**CONJECTURAL**).
- The needed bridge lemma — swapping adjacent independent actions preserves lawfulness — is stated with its exact five-part hypothesis (**CONJECTURAL**). This is the technical crux: it is where PDDL 3.1’s temporal, numeric, and trajectory constraints must actually be discharged.

6.3 Diagnostics: how you would check maximality on real output

Even before the conjecture is settled, maximality is *auditable*, statistically, on any concrete converted model:

- **Under-merging test** (missed concurrency): find behaviors $b_1 \sim_I b_2$ with $\tau(b_1) \neq \tau(b_2)$. Each is a certificate that an edge of the emitted partial order is not causally forced.
- **Over-merging test** (false concurrency): find $\tau(b_1) = \tau(b_2)$ with $b_1 \not\sim_I b_2$ — a soundness bug.
- **Entropy audit**: compare $\log |F_w|$ (from samples) with $\log e(P_w)$ (computable from the emitted order). Equality is the fingerprint of maximality; a deficit measures surviving accidental order, in nats.
- **Tension audit**: the formalization defines per-aspect “generalized dimensions” D_q and flags patterns such as $D_q^{\text{Linearization}} \gg D_q^{\text{Temporal}} = \text{false concurrency pressure}$: the diagram advertises interleavings the clock forbids. This is the executable-vs-causal gap made measurable.

7 The verdict

Q1 — Can it be converted? Not unconditionally, but yes on a precisely characterized and practically dominant class. If the PDDL 3.1 model’s reachable behavior forms a safe and sound *separable* workflow net (decisions and concurrency nest, never cross-link), conversion to POWL v2 is guaranteed (**PROVEN**, Kourani et al. Thm. 5.6), and language-preserving preprocessing widens the class. Outside it there are genuine counterexamples — long-term cross-choice dependencies, loops jumping into parallel branches, choice/concurrency interleaved at a shared synchronization — for which *no* POWL 2.0 model exists. Empirically, 100% of 1,493 industrial/synthetic process models converted; planning domains with heavy cross-branch coupling are exactly where to expect the exceptions.

Q2 — Completely? Yes, whenever Q1 succeeds: $\mathcal{L}(N) = \mathcal{L}(\psi)$ ([PROVEN](#), Thm. 5.5). The POWL v2 model permits exactly the original execution sequences. One scoping honesty: this equality is about *orderings of actions*. PDDL 3.1’s metric time, numeric fluents, and trajectory constraints must either be discharged before conversion (only lawful behavior enters the net) or carried alongside the POWL model; the diagram itself does not repeat them.

Q3 — With concurrency maximized? This is exactly the MFW crown theorem, $\ker(\tau) = \text{TraceEq}_I$: the conversion merges all of — and only — the behaviors that differ by reordering independent actions. Status: [CONJECTURAL](#), with the surrounding scaffolding (equivalence structure, fiber partition/disjointness, non-circularity, the conditional fiber-equals-trace-class identity) machine-checked in Lean 4. Two qualifications are permanent, not gaps: (i) maximality is relative to the independence relation I — an over-cautious I over-sequentializes safely, an over-eager I is unsound, so “maximal” means *maximal given the true I* ; (ii) PDDL 3.1’s clocks and resources make maximal *causal* concurrency an overestimate of *executable* concurrency $C_E = C_C \cap C_T \cap C_R$; the temporal restriction gap $H_{\text{ser}} - H_T \geq 0$ measures the difference. Maximality *is* auditable today, per-model, by the entropy test $\log |F_w| \stackrel{?}{=} \log e(P_w)$.

One-sentence answer. For the separable class — which provably admits a complete, behavior-exact conversion and which covered every model in a large industrial benchmark — yes; and “concurrency maximized” has an exact formal meaning (erase all and only accidental order) that is conjectured, machine-checkably scaffolded, and per-model auditable, with the permanent caveat that PDDL 3.1’s temporal and resource constraints cap *executable* concurrency below causal concurrency.

Glossary

Lawful behavior Action+timestamp sequence satisfying transitions, goal, and all PDDL 3.1 constraints.

$\mathcal{P}(\Pi)$ Sample space of all lawful behaviors.

Independence I Pairs of actions that commute in effect, preconditions, invariants, numerics, and trajectory constraints.

Trace equivalence $\sim_I$ Reachability by adjacent swaps of independent actions; “same story, different typing order.”

Causal order $P_b; e(P)$ The forced precedences of a behavior; the count of its linear extensions.

Serialization entropy $H_{\text{ser}} = \log e(P)$ How much accidental order a sequence representation carries.

WF-net; safe; sound; separable Petri net with unique start/end; one-token places; no dead parts and guaranteed completion; recursively built from state machines (choice) and marked graphs (concurrency).

POWL v2 Hierarchical models from partial orders (concurrency) and choice graphs (decisions/cycles); language $\mathcal{L}(\psi)$.

τ ; **fiber** F_w ; **kernel** The conversion map; the set of behaviors merged into workflow w ; the induced equivalence.

Fiber entropy $S_\tau(w) = \log |F_w|$ Information erased by the conversion at w .

Crown theorem $\ker(\tau) = \text{TraceEq}_I$ — the formal statement of “complete conversion with concurrency maximized.”

References

- [1] H. Kourani, G. Park, W. M. P. van der Aalst, *Hierarchical Decomposition of Separable Workflow-Nets*, arXiv:2602.15739v3, 2026. (Correctness Thm. 5.5, Completeness Thm. 5.6, counterexamples Figs. 2 & 7, benchmark Table 1.)
- [2] H. Kourani, S. J. van Zelst, *POWL: Partially Ordered Workflow Language*, BPM 2023.
- [3] A. Mazurkiewicz, *Concurrent program schemes and their interpretations*, DAIMI PB-78, 1977; V. Diekert, G. Rozenberg (eds.), *The Book of Traces*, World Scientific, 1995.
- [4] `ProcInt.MFW`, Lean 4 formalization (toolchain v4.31.0), `procint/ProcInt/MFW`; companion notation paper `paper/mfw-notation.tex`.
- [5] M. Fox, D. Long, PDDL2.1/3.x temporal and trajectory extensions of the Planning Domain Definition Language.